

Flight events from ADS-B: fixing what OPDI publishes, and reaching the GANP KPIs

Version 2 — with the extraction methodology for every event

EUROCONTROL Performance Review Unit — Open Performance Data Initiative

2026-08-15

OPDI has published flight events since v0.0.2 without a single test or a single comparison against reference data. This study audits the four detectors behind them, finds nine defects — one of which silently suppresses every take-off and landing at any aerodrome above roughly 200 ft — fixes them, and adds four milestone families chosen because they are what the ICAO GANP key performance indicators actually require. It reports what has been measured and, equally, what has not. Version 2 adds a full account of how each of the thirty-four published event types is extracted: the algorithm, its parameters and where they come from, the code that implements it, and what is known to be wrong with it.

Table of contents

1	What changed in version 2	2
2	What this study is	2
3	What was wrong	3
4	What was built	4
5	Which KPIs this reaches	4
6	Ground truth, and its ceiling	5
7	How each event is extracted	6
7.1	1. Flight phase classification	6
7.2	2. take-off and landing	7
7.3	3. top-of-climb and top-of-descent	7
7.4	4. level-start and level-end (phase-derived)	7
7.5	5. first-xing-fl* and last-xing-fl* — the published crossings	8
7.6	6. xing-fl150, xing-fl170, xing-fl100, xing-fl245	8
7.7	7. xing-40nm and xing-100nm — the ASMA rings	9
7.8	8. entry-* and exit-* — airport layout events	9
7.9	9. ATOT and ALDT — runway identification and touchdown	10
7.10	10. AOBT and AIBT — block times	11
7.11	11. ICAO level segments	11
7.12	first_seen and last_seen	12

7.13 Measurements	12
7.14 Event identity	13
8 What the detectors produce	13
9 How accurate the milestones are	14
10 What the numbers say	15
10.1 Landing time is transformed; take-off trades sharpness for reach	15
10.2 Requiring complete evidence made detection worse	15
10.3 Off-block and on-block are limited by reception, as expected	16
10.4 Does it hold on a second period?	16
10.5 What has still not been measured	16
11 Breaking change for consumers	17
12 Provenance	17

1 What changed in version 2

Version 1 reported *what* the detectors produce and how accurate the four milestones APDF can score turned out to be. It never said *how* any event is extracted, which left a reader unable to tell what triggers a **top-of-climb**, what **entry-runway** matches against, or which parameter would change either.

That gap mattered more than it might elsewhere, because this study’s findings are about algorithm *shape* — that requiring complete evidence from a fuzzy rule is harmful, that a smoothing window behaves differently on noisier input, that a threshold detector is unusable without a dead band. None of those is legible without the method beside it.

Version 2 adds that account and changes nothing else. **The data, the figures and the findings are version 1’s**, carried across with their provenance records intact rather than regenerated.

2 What this study is

Two things prompted it.

The events were never checked. OPDI’s flight-event table is the least-examined thing the project publishes. Departure and destination aerodrome detection has been through seven successive studies and a full rewrite; the event detectors had no tests at all, and no number in them had ever been compared against anything.

The event vocabulary did not reach the indicators. The stated goal is to compute as many ICAO GANP key performance indicators as ADS-B allows. Mapping the published KPI list against what OPDI emitted showed the gap concentrated in four milestone families — and that ICAO itself names ADS-B data providers as a feed for two indicators OPDI could not compute at all.

3 What was wrong

Nine defects, found by reading the detectors. None of them fails. Each produces fewer events, or events displaced in one direction — the failure mode that survives an eyeball check of the totals.

	Defect	Consequence
D1	Ground membership measured against uncorrected pressure altitude	The membership reaches zero at 200 ft <i>pressure</i> altitude, so at any aerodrome above roughly 200 ft AMSL — or anywhere on a low-pressure day — no sample is ever classified as on-ground. Since take-off requires a preceding ground sample and landing requires a ground sample, neither event has ever been emitted for those flights . The loss is biased against high-elevation aerodromes.
D2	The reference implementation's 60-second majority smoothing was not ported	Phases were decided per state vector at 5-second spacing with no temporal aggregation, so one misclassified sample injects a spurious level segment or destroys a take-off.
D3	Cruise speed membership centred on 600 kt	A turboprop at 250 kt scores 0.002, so it never reaches cruise and gets no top-of-climb or top-of-descent. Inherited from OpenAP, not introduced here — left unchanged pending measurement.
D4	least/greatest skip nulls	A rule missing an input became a minimum over fewer terms, which can only <i>raise</i> its activation — so incomplete evidence out-competed complete evidence.
D5	first/last inside a group-by	Take partition order, not time order, so a reported entry position was not guaranteed to be the position at the reported entry time.

	Defect	Consequence
D6	Events read the raw track table	Trajectory cleaning reached the flight list and nothing else, so no published event has ever been derived from a cleaned trajectory.
D7	Non-reproducible event identifiers; one version string for every type	The same event got a different identifier on every run, so two runs of a month could not be reconciled.
D8	Flight-level crossings kept only the first and last per level, uninterpolated	A flight that levels off and re-climbs through a level lost every crossing in between — exactly the crossings a vertical-efficiency indicator is about — and every reported instant was biased by up to one sample interval in the same direction.
D9	No tests	Not one detector was covered.

D1 and D2 are the two that matter most, because they mean the published take-off, landing, top-of-climb and top-of-descent events are missing **non-randomly**.

4 What was built

Eight defects are fixed and the ninth (D3) is left in place deliberately, because changing a constant inherited from the reference implementation should follow a measurement rather than precede one. Four milestone families were added:

- **Threshold crossings**, rewritten. Every crossing rather than the first and last, with a hysteresis dead band so an aircraft cruising *at* a level emits none, and the instant interpolated to the exact threshold.
- **ASMA ring crossings** at 40 and 100 NM, built around the flight’s own origin and destination — which is what the indicators mean, and what the reference data records.
- **ICAO level segments**, implementing the published algorithm rather than reusing the fuzzy-phase level events. The two are **not interchangeable**.
- **Runway identification with ATOT and ALDT**, and **off-block/on-block**.

Every change is reversible: a `legacy()` configuration reproduces published `events_v0.0.2` exactly, and the new algorithm carries a new version string.

5 Which KPIs this reaches

KPI	Title	Reachable from ADS-B?
KPI02	Taxi-out additional time	Yes

KPI	Title	Reachable from ADS-B?
KPI05	Actual en-route extension	Yes — ICAO names ADS-B as the feed
KPI08	Additional time in terminal airspace	Yes
KPI10	Airport peak throughput	Yes
KPI13	Taxi-in additional time	Yes, coverage-limited
KPI15	Flight time variability	Yes
KPI17	Level-off during climb	Yes
KPI18	Level capping during cruise	Yes (variant 2)
KPI19	Level-off during descent	Yes
KPI01, KPI14	Departure / arrival punctuality	Actual half only — the scheduled half needs a schedule feed
KPI11	Airport throughput efficiency	Numerator only
Others	ATFM, capacity, flight plan, fuel, safety, noise	No

Nine fully and three partially, from four milestone families.

6 Ground truth, and its ceiling

Reference data is EUROCONTROL APDF. It is in movement form — there is no literal take-off-time column, and `SRC_PHASE` decides what a row carries — and crucially it has **no aircraft address**, so it cannot be joined to ADS-B directly. The route runs through the flight table on an internal key.

That bridge is the ceiling on every coverage figure in this study: a milestone that cannot be reached is indistinguishable from one that was missed, unless the ceiling is known. So it is measured first.

Table 1

Period	APDF movements	Reaching icao24	Share
2024-06	1,161,115	1,087,091	93.62%
2025-06	1,224,742	1,154,338	94.25%

Both periods clear 93%, so the ceiling is high enough not to dominate any result. The residual loss is roughly three per cent of movements carrying no internal key, and a further two per cent whose key has no counterpart in the flight table.

Two traps in this join are worth recording, because both fail silently:

The reference carries the commercial flight number, ADS-B carries the ATC call-sign. The two disagree on 20.4% of rows — LH416 against DLH416 — and joining on the wrong one discards a fifth of the sample while looking exactly like a detection failure.

Timestamps are tagged Europe/Paris despite being named UTC. The instants are correct, so converting is right and stripping the zone is a two-hour error. Spark’s session timezone is set to UTC in the local session builder and not in the distributed one, so the loader asserts it rather than assuming it.

7 How each event is extracted

Every event OPDI publishes is produced by one of eleven detectors. This section states, for each, what the event means, how it is found, which parameters govern it and where they come from, which module implements it, and what is known to be wrong with it.

Three conventions apply throughout. **Storage is SI and thresholds are aviation** — altitudes are metres and vertical rates metres per second on disk, scaled at the point of comparison into feet and feet per minute, because those are the units the thresholds are stated in. **Every threshold is named** in `EventConfig` (`src/opdi/config.py`); none is an inline literal, so any of them can be swept by the benchmark. And `EventConfig.legacy()` reproduces `events_v0.0.2`, so the detectors below describe the new version while the published one remains reachable.

7.1 1. Flight phase classification

Not an event type itself: it is the substrate for take-off, landing, top of climb and descent, and the phase-derived level segments.

Each state vector is assigned a phase — ground, climb, descent, cruise or level — by fuzzy logic. Nine membership functions score altitude, rate of climb and speed; five rules combine them with a minimum t-norm; the phase is the rule with the highest activation.

Membership	Function	Parameters
ground altitude	zmf	$0 \rightarrow 200$ ft
low altitude	gaussmf	μ 10,000 ft, 10,000
high altitude	gaussmf	μ 35,000 ft, 20,000
zero climb rate	gaussmf	μ 0 ft/min, 100
positive climb rate	smf	$10 \rightarrow 1,000$ ft/min
negative climb rate	zmf	$-1,000 \rightarrow -10$ ft/min
high speed	gaussmf	μ 600 kt, 100
medium speed	gaussmf	μ 300 kt, 100
low speed	gaussmf	μ 0 kt, 50

These constants are OpenAP’s published values, not chosen here. That matters for one known limitation: a turboprop cruising at 250 kt scores 0.002 on the high-speed membership, so the cruise rule never wins and **turboprop and general-aviation traffic get no top-of-climb or top-of-descent at all**. It is inherited from the reference implementation rather than an OPDI defect, and it is left in place deliberately — changing a reference constant should follow a measurement rather than precede one.

Two things differ from a naive port:

A 60-second majority smoothing (`phase_twindow_seconds`) is applied to the per-sample labels. OpenAP calls `phaselabel(twindow=60)`; the original OPDI port carried the membership functions across and dropped the smoothing, so phases were decided per state vector at 5-second spacing with no temporal aggregation. One misclassified sample was enough to inject a spurious level segment or destroy a take-off. Ties keep the unsmoothed label, which makes this a de-flicker rather than a relabel. **This study found the change is not robust across periods** — see the results section.

Ground membership is measured above field elevation (`phase_ground_above_field`). `baro_altitude_c` is uncorrected pressure altitude, so a 200 ft ceiling is unreachable at any

aerodrome above roughly 200 ft AMSL or on a low-pressure day anywhere. The elevation of **both** the track’s ADEP and ADES is attached and the more permissive membership taken: a track is on the ground at only one of its ends, and cruise sits far above both, so the wrong end can only contribute a zero. A missing elevation falls back to zero, which is the published behaviour rather than a loss of phases.

Implemented in pipeline/events.py: calculate_horizontal_segment_events, _smooth_phase, attach_field_elevation.

7.2 2. take-off and landing

The instant the aircraft leaves or meets the ground, as the phase sequence sees it.

take-off fires where the previous sample was ground and the next is climb. **landing** fires where the previous sample was descent and the current is ground. The two definitions are deliberately asymmetric — one looks forward, one does not — which is inherited from the published detector and preserved so the type keeps its meaning.

Known weaknesses. Both depend entirely on a ground sample existing, so sparse surface reception removes them silently. Both are measured here against APDF and shown to have low coverage (1.8% and 2.1%) with a very long tail — the 90th-percentile take-off error is 71 minutes. The runway-anchored ATOT/ALDT below are the intended replacement; these keep their published type strings so existing consumers are undisturbed.

Implemented in pipeline/events.py: calculate_horizontal_segment_events.

7.3 3. top-of-climb and top-of-descent

The first and last sample classified as cruise, per track.

Simple by construction, and therefore inheriting everything above: a flight that never reaches the cruise rule — a turboprop, or anything with missing velocity — has neither event. **Neither has ever been validated against an external source**, because none exists that records them.

Implemented in pipeline/events.py: calculate_horizontal_segment_events.

7.4 4. level-start and level-end (phase-derived)

The boundaries of a run of samples classified cruise or level.

A **level-start** is emitted where the phase enters cruise or level from something else, a **level-end** where it leaves. The first cruise sample carries both **level-start** and **top-of-climb**; the last carries **level-end** and **top-of-descent**.

These are not the ICAO level segments in section 11, and the two are not interchangeable. This pair asks “does this sample look like level flight” through the fuzzy classifier; ICAO asks a geometric question about an altitude band anchored at a segment’s own start. Both are published, under different names.

Their volume is the clearest single measure of phase flicker: 1,270,412 **level-start** events under the published detector against 370,466 under the new one, on the same three days.

Implemented in pipeline/events.py: calculate_horizontal_segment_events.

7.5 5. first-xing-fl* and last-xing-fl* — the published crossings

The first and last time a track crosses FL50, FL70, FL100 or FL245.

A crossing is detected by comparing each sample's flight level with the next one's; the sample *before* the crossing is reported. A guard requires the sample to be within 10 flight levels of the boundary, which rejects crossings inferred across a data gap. The first and last per level are kept and everything between is discarded.

Three faults, which is why section 6 replaces this. Keeping only first and last discards every crossing in between — a flight that levels off and re-climbs through FL100 loses exactly the crossings a vertical-efficiency indicator is about. Reporting the bracketing sample biases every timestamp by up to one sample interval, always in the same direction. And the level is chosen by a first-match chain, so a single 5-second gap spanning FL50 *and* FL70 reports only one of them.

These types are emitted only under events_v0.0.2. The new version does not re-emit them; consumers migrate to `crossing_seq` in section 6.

Implemented in `pipeline/events.py`: `calculate_vertical_crossing_events`.

7.6 6. xing-fl50, xing-fl70, xing-fl100, xing-fl245

Every crossing of each monitored flight level, with the instant interpolated to the level itself.

The detector is a Schmitt trigger. For a level L and dead band h , a sample is assigned $+1$ above $L + h$, -1 below $L - h$, and null inside the band. That side is forward-filled, and a crossing is confirmed where the filled value flips. An aircraft cruising *at* the level never leaves the band, so its side never resolves and **no event is emitted** — without this, barometric noise at a cruise level produces a crossing on almost every sample.

The consequence has to be stated with the benefit: **a flight that climbs and levels off exactly at FL100 registers no FL100 crossing**, because it never clears the upper edge. That case is described by the level-segment and top-of-climb events instead.

The instant is then interpolated to exactly L **between the two samples that straddle it** — which is a different row from the one that confirmed the crossing. Confirmation happens at the band edge; the crossing itself is earlier. Conflating them reintroduces the one-sided bias this replaces. Each event carries `crossing_seq` (1-based, in time order), `direction`, and `bracket_s`, the spacing of the two interpolated-between samples, so a consumer can tell an instant measured across 5 seconds from one inferred across four minutes.

Parameter	Value	Source
<code>crossing_levels_fl</code>	50, 70, 100, 245	the published vocabulary
<code>crossing_hysteresis_ft</code>	300 ft	ours
<code>crossing_all_occurrences</code>	on	new
<code>crossing_interpolate</code>	on	new

Thresholds are evaluated as parallel columns over one window specification rather than by exploding the trajectory once per level, so four levels cost one shuffle rather than four, and only rows where a crossing fired are expanded into events.

Implemented in `pipeline/crossings.py`: `threshold_crossings`, `flight_level_crossings`; wired by `events.calculate_threshold_crossing_events`.

7.7 7. xing-40nm and xing-100nm — the ASMA rings

Crossings of the 40 NM and 100 NM cylinders around the flight’s **own** origin and destination.

The same detector as section 6, with distance to an aerodrome as the quantity instead of altitude, and a dead band of 1 NM. The rings come from the flight list’s ADEP and ADES rather than from the H3 aerodrome zone table, for two reasons: it is what the indicators mean — KPI08’s ASMA is a cylinder around the destination and KPI05’s reference area one around each end — and it is what APDF records, one crossing per movement rather than one per aerodrome overflown. It is also far cheaper, multiplying each sample by at most two rather than by every aerodrome within 110 NM.

`info` carries `crossing_seq`, `direction` (inbound/outbound), `apt_icao`, `bearing` and `flight_level`, matching `C40_CROSS_{TIME,LAT,LON,FL}` and `C40_BEARING` column for column so the comparison against reference data is direct rather than a translation.

Bearing is computed from the interpolated crossing position, not interpolated alongside it. A bearing is circular: averaging 359 and 1 gives 180, which would be wrong by half a turn on any crossing near due north.

A bug worth recording, because it shaped this section. The first run produced 2.85 million 40 NM crossings and 3.20 million at 100 NM against 236,899 tracks — with *identical* counts for both rings on the worst flights, a signature real geometry cannot produce. The cause was in a shared helper: `haversine_nm` clamped the argument of `asin` with `F.least(a, 1.0)`, and `F.least` skips nulls, so a missing coordinate became `asin(1)` — 10,807 NM, the antipodal distance. A transponder parked at Budapest transmitted for the whole 72-hour sample; cleaning nulled its repeated positions as stale broadcasts, and its distance series then alternated between 0.7 NM and the antipode, crossing both rings on nearly every sample. After the fix the counts are 151,526 and 138,512, a 19- and 23-fold reduction.

Implemented in `pipeline/crossings.py`: `ring_crossings`; wired by `events.calculate_ring_crossing_eve`

7.8 8. entry-* and exit-* — airport layout events

Types emitted: `entry-runway`, `exit-runway`, `entry-taxiway`, `exit-taxiway`, `entry-apron`, `exit-apron`, `entry-hangar`, `exit-hangar`, `entry-threshold`, `exit-threshold`, `entry-parking_position`, `exit-parking_position`, `entry-deicing_pad`, `exit-deicing_pad`.

Entry to and exit from runway, taxiway, apron, hangar, threshold, parking position and de-icing pad.

Low-altitude samples are matched against OpenStreetMap aeroway geometry that has been pre-rasterised to H3 resolution 12 hexagons in `hexaero_airport_layouts`. A sample joins a feature when its `h3_res_12` cell matches and the feature’s aerodrome is one the flight list names for that flight. Consecutive matches on the same feature form a traversal; a gap of more than 300 seconds starts a new one. Each traversal yields an entry and an exit event.

Parameter	Value	Source
<code>airport_max_fl</code>	FL20	the published gate
<code>airport_trace_gap_seconds</code>	300 s	the published gate
H3 resolution	12	<code>H3Config.airport_layout_resolution</code>

Entry and exit attributes are taken from the samples at the entry and exit times (`airport_events_ordered`). The published detector used `first/last` inside a group-by, which take partition order rather than time order — so a reported entry position was not guaranteed to be the position at the reported entry time. This is a correctness fix, not a tuning knob.

Two limitations that bound coverage. The detector cannot fire for an aerodrome the flight list did not name, and it depends on OSM coverage, which `hexaero_airport_layouts` builds only for large and medium aerodromes within the ingestion area. A traversal completed inside a single 5-second sample is also dropped, because its duration is zero.

`entry-threshold` and `exit-threshold` fall from 774 to 9 between the published and new configurations. The threshold polygons are the smallest features in the layout set, so they are the most sensitive to which samples survive cleaning — this is the clearest instance of trajectory cleaning removing the samples a detector depends on, and it is not a detector change.

Implemented in `pipeline/events.py`: `calculate_airport_events`.

7.9 9. ATOT and ALDT — runway identification and touchdown

Take-off and landing times, anchored on the runway the movement actually used.

A native port of `traffic`'s `TrackBasedRunwayDetection`. Samples are kept when they are within 5 NM of the aerodrome, below 1,500 ft above field elevation, faster than 30 kt over the ground, and climbing or descending faster than 257 ft/min in the direction the role implies. At least four must survive. The median track of the survivors is matched against each runway's bearing, and the nearest within 10 degrees wins; parallel runways are separated by cross-track distance to each centreline, in closed form where `traffic` uses a geometry library.

Runway geometry comes from `oa_runways` — the `OurAirports` table step 00d has been generating since it was written and which nothing had ever read. It is the same data `traffic` downloads at runtime, so this adds no dependency and nothing to warm on an offline executor. **Bearings are computed from the two threshold positions** rather than taken from `le_heading_degT`, which is often null and sometimes magnetic rather than true.

Parameter	Value	Source
<code>runway_max_dist_nm</code>	5 NM	<code>traffic</code>
<code>runway_max_height_ft</code>	1,500 ft	<code>traffic</code>
<code>runway_min_vert_rate_ftmin</code>	257 ft/min	<code>traffic</code>
<code>runway_min_airspeed_kt</code>	30 kt	<code>traffic</code>
<code>runway_max_bearing_deg</code>	10°	<code>traffic</code>

The detector **abstains** when no runway falls within the bearing tolerance. Naming the wrong runway corrupts a movement count; naming none is an explicit null.

Both times are proxies, and are documented as such: ATOT is the earliest surviving sample of a departure and ALDT the latest of an arrival. Their bias against reference data is measured and reported in the results rather than assumed to be zero.

Implemented in `pipeline/runways.py`: `runway_thresholds`, `detect_runway_movements`; wired by `events.calculate_runway_events`.

7.10 10. AOBT and AIBT — block times

Off-block and on-block.

The aircraft is moving once its groundspeed stays above 2 kt for a sustained 30 seconds — a port of the signal in `traffic`’s `StartMoving`, where the sustained part is what separates a push from a jitter in the speed field. The start of the first such run is the off-block candidate and the end of the last the on-block candidate.

Both are **anchored on the parking-position events** from section 8: a candidate is only emitted for a track that was actually seen leaving or entering a stand. Without that anchor, a track first received mid-taxi would report its first movement as an off-block — a different event, and one that reads as an implausibly short taxi rather than as a miss.

`traffic`’s own parking-position path is deliberately **not** used: it resolves stands through a live OpenStreetMap Overpass query whose cache does not survive serialisation to an executor, so it cannot run without network access. OPDI’s `hexaero_airport_layouts` already carries the same geometry.

Parameter	Value	Source
<code>ground_speed_threshold_kt</code>	2 kt	<code>traffic</code>
<code>ground_move_min_seconds</code>	30 s	<code>traffic</code>

Coverage is the limitation, and it is measured: 2.6% of reference movements, with the off-block proxy firing around twelve minutes late. Surface reception is sparse, and sustained movement is detectable only once reception begins. Both numbers are reported as the finding rather than tuned away, and they are the reason pushback (T05) was not built — it needs strictly more coverage than this.

Implemented in `pipeline/ground.py`: `movement_window`, `block_times`; wired by `events.calculate_block_e`

7.11 11. ICAO level segments

Types emitted: `level-off-climb-start`, `level-off-climb-end`, `level-off-descent-start`, `level-off-descent-end`.

Level flight during climb and descent, as ICAO defines it for KPI17 and KPI19.

ICAO specifies these with a complete algorithm rather than leaving detection to the implementer, and this is that algorithm:

a data point starts a level segment when the altitude difference with the next data point is the level band limit and the vertical speed towards it is the vertical speed limit. The segment ends when the altitude differs from the segment’s starting altitude by more than the level band limit, or the vertical speed between two consecutive points exceeds the limit.

The anchoring is what a plain window function cannot express: a segment’s end is defined against *its own start*, a running value that resets. It is implemented as a sessionisation — mark where flatness begins, forward-fill the anchor within the group — and each group is truncated at its first failure, so a trajectory that leaves the band and drifts back is not stitched across the excursion. Anchoring against the segment start rather than the previous sample is what stops a slow drift being read as one long level segment.

Segments are then split into KPI17 and KPI19 by whether they fall before top-of-climb or after top-of-descent, with two exclusions: a **minimum altitude** below which the trajectory is not analysed, and an **exclusion box** that removes a segment sitting above 90% of the cruise altitude and lasting longer than five minutes — that is cruise, not a level-off, and without the box every cruise would be counted as the largest level-off in the flight.

Parameter	Value	Source
level_band_limit_ft	200 ft	ICAO
level_vertical_speed_limit_ftmin	300 ft/min	ICAO
level_min_duration_seconds	20 s	ICAO
level_exclusion_box_pct	90%	ICAO
level_exclusion_box_seconds	300 s	ICAO
level_min_altitude_climb_ft	3,000 ft	ICAO
level_min_altitude_descent_ft	1,800 ft	ICAO
level_analysis_radius_nm	200 NM	ICAO

Top-of-climb and top-of-descent are taken from the phase detector’s own output rather than recomputed, so the two families cannot disagree about where cruise began — which matters because the exclusion box is defined against the cruise altitude.

No source available to OPDI holds level-segment truth, so the claim made for this family is conformance to the published algorithm, tested on synthetic trajectories whose geometry is known by construction. That is checkable; “accurate” is not.

Implemented in `pipeline/level_segments.py`: `level_segments`, `classify_level_offs`; wired by `events.calculate_level_off_events`.

7.12 first_seen and last_seen

The first and last sample of each track, by time. Emitted for every track without condition, which is why their count — 236,899 on the sample — is also the track count.

Implemented in `pipeline/events.py`: `calculate_firstseen_lastseen_events`.

7.13 Measurements

Two values are attached to every event rather than being events themselves.

Distance flown (NM) is the cumulative great-circle distance from the start of the track, summed sample to sample with an Earth radius of 6,371 km and converted at 1.852 km per nautical mile. **Time Passed (s)** is the elapsed time from the track’s first sample.

Both are **interpolated to the crossing instant** for the crossing families rather than read from a neighbouring sample — a measurement attached to an interpolated event but taken from a different row would disagree with its own event.

Implemented in `pipeline/events.py`: `add_distance_measure`, `add_time_measure`.

7.14 Event identity

Each event’s identifier is derived from the event’s own identity — track, type, time and version — rather than from a partition counter. The published detector used `monotonically_increasing_id()`, which encodes the partition index, so the same event received a different identifier on every run and two runs of a month could not be reconciled.

This makes a re-run idempotent in *content*. It does **not** make the write idempotent: the storage layer appends, and the event tables are partitioned by (**type**, **version**) rather than by month, so an overwrite would destroy every other month. Re-processing safety currently rests on the per-month progress logs, and a month partition on the published tables is recorded as deferred work.

8 What the detectors produce

Every event type step 04 emits, counted on the sample. Most of these have no counterpart in any reference dataset, so this table is the only place they are visible at all — a family that stopped being emitted would otherwise show up not as a regression but as an absence, which is harder to notice.

Table 2

Event type	L00_legacy
AIBT	—
ALDT	—
AOBT	—
ATOT	—
entry-apron	2,565
entry-hangar	34
entry-parking_position	6,531
entry-runway	31,233
entry-taxiway	30,934
entry-threshold	774
exit-apron	2,564
exit-hangar	34
exit-parking_position	6,479
exit-runway	31,175
exit-taxiway	29,433
exit-threshold	774
first-xing-fl100	95,845
first-xing-fl245	86,384
first-xing-fl150	98,017
first-xing-fl170	96,633
first_seen	236,899
landing	9,232
last-xing-fl100	95,845
last-xing-fl245	86,384
last-xing-fl150	98,017
last-xing-fl170	96,633
last_seen	236,899

Event type	L00_legacy
level-end	888,611
level-off-climb-end	—
level-off-climb-start	—
level-off-descent-end	—
level-off-descent-start	—
level-start	1,270,412
take-off	8,483
top-of-climb	114,099
top-of-descent	108,767
xing-100nm	—
xing-40nm	—
xing-fl100	—
xing-fl245	—
xing-fl50	—
xing-fl70	—

9 How accurate the milestones are

Scored against APDF on the milestones it can reach. Coverage is the share of reference milestones with any detection; bias is the median signed error, so a negative value means the detector fires early.

Table 3

OPDI			Truth	Detected	Coverage	Bias (s)	MAD (s)	±60 s
Rung	type	vs						
L11_block	climb	AIBT	57,362	1,520	2.65%	-323.0	323.0	0.01%
L12_level	climb	AIBT	57,362	1,520	2.65%	-323.0	323.0	0.01%
L13_ship	climb	AIBT	57,362	1,520	2.65%	-323.0	323.0	0.01%
L00_legacy	climb	ALDT	57,363	1,215	2.12%	34.0	35.0	1.61%
L01_clear	climb	ALDT	57,363	3,668	6.39%	7.0	14.0	5.41%
L02_smooth	climb	ALDT	57,363	3,724	6.49%	6.0	12.0	5.59%
L03_complete	climb	ALDT	57,363	393	0.69%	22.0	22.0	0.67%
L04_field	climb	ALDT	57,363	1,066	1.86%	21.0	21.0	1.78%
L05_order	climb	ALDT	57,363	1,066	1.86%	21.0	21.0	1.78%
L06_determine	climb	ALDT	57,363	1,066	1.86%	21.0	21.0	1.78%
L07_all	climb	ALDT	57,363	1,066	1.86%	21.0	21.0	1.78%
L08_inter	climb	ALDT	57,363	1,066	1.86%	21.0	21.0	1.78%
L09_ring	climb	ALDT	57,363	1,066	1.86%	21.0	21.0	1.78%
L10_run	climb	ALDT	57,363	49,366	86.06%	-2.0	19.0	80.38%
L10_run	climb	ALDT	57,363	1,066	1.86%	21.0	21.0	1.78%
L11_block	climb	ALDT	57,363	49,366	86.06%	-2.0	19.0	80.38%
L11_block	climb	ALDT	57,363	1,066	1.86%	21.0	21.0	1.78%
L12_level	climb	ALDT	57,363	49,366	86.06%	-2.0	19.0	80.38%
L12_level	climb	ALDT	57,363	1,066	1.86%	21.0	21.0	1.78%
L13_ship	climb	ALDT	57,363	49,366	86.06%	-2.0	19.0	80.38%
L13_ship	climb	ALDT	57,363	1,066	1.86%	21.0	21.0	1.78%
L11_block	climb	AOBT	57,363	1,509	2.63%	744.0	748.0	0.00%

Rung	OPDI		Truth	Detected	Coverage	Bias (s)	MAD (s)	± 60 s
	type	vs						
L12_level	AOBT	AOBT	57,363	1,509	2.63%	744.0	748.0	0.00%
L13_ship	AOBT	AOBT	57,363	1,509	2.63%	744.0	748.0	0.00%
L00_legacy	take-off	ATOT	57,364	1,041	1.81%	1.0	7.0	1.31%
L01_clear	take-off	ATOT	57,364	1,871	3.26%	4.0	11.0	2.42%
L02_small	take-off	ATOT	57,364	688	1.20%	-1.0	12.0	1.15%
L03_comp	take-off	ATOT	57,364	20	0.03%	-21.0	23.0	0.03%
L04_field	take-off	ATOT	57,364	223	0.39%	-1.0	16.0	0.38%
L05_order	take-off	ATOT	57,364	223	0.39%	-1.0	16.0	0.38%
L06_detect	take-off	ATOT	57,364	223	0.39%	-1.0	16.0	0.38%
L07_all	take-off	ATOT	57,364	223	0.39%	-1.0	16.0	0.38%
L08_inter	take-off	ATOT	57,364	223	0.39%	-1.0	16.0	0.38%
L09_rings	take-off	ATOT	57,364	223	0.39%	-1.0	16.0	0.38%
L10_run	ATOT	ATOT	57,364	36,313	63.30%	19.0	21.0	58.63%
L10_run	take-off	ATOT	57,364	223	0.39%	-1.0	16.0	0.38%
L11_block	ATOT	ATOT	57,364	36,313	63.30%	19.0	21.0	58.63%
L11_block	take-off	ATOT	57,364	223	0.39%	-1.0	16.0	0.38%
L12_level	ATOT	ATOT	57,364	36,313	63.30%	19.0	21.0	58.63%
L12_level	take-off	ATOT	57,364	223	0.39%	-1.0	16.0	0.38%
L13_ship	ATOT	ATOT	57,364	36,313	63.30%	19.0	21.0	58.63%
L13_ship	take-off	ATOT	57,364	223	0.39%	-1.0	16.0	0.38%

10 What the numbers say

10.1 Landing time is transformed; take-off trades sharpness for reach

Both the published fuzzy-phase pair and the new runway-anchored pair are scored against the same APDF truth, because which of them wins is the question this study exists to answer.

The published **landing** is beaten on every axis at once: the new **ALDT** covers forty times as many flights, its median error is two seconds early rather than thirty-four late, and four in five detections fall within a minute against one in sixty.

take-off is the more interesting case, and the honest reading is a trade rather than a win. The published detector has the *better median* — a one second bias against nineteen — but it fires on 1.8% of flights and its 90th percentile error is **4,273 seconds**. Seventy-one minutes. So it is often excellent and occasionally absurd, and nothing in its output distinguishes the two. The new **ATOT** covers thirty-five times more flights with a 90th percentile of 59 seconds. For an indicator averaged over a population, a detector that is never wildly wrong is worth more than one that is sometimes perfect.

10.2 Requiring complete evidence made detection worse

The ladder’s purpose is attribution, and it earned that here by contradicting one of this study’s own fixes.

least/greatest skip nulls, so a fuzzy rule missing an input became a minimum over fewer terms and could out-compete a complete rule. Requiring every input to be present is obviously more correct. On real data it removes 90% of phase-based landings — coverage falls from 6.49% to

0.69% — because ADS-B carries a null velocity often enough that insisting on complete evidence discards most ground samples.

It matters less than it appears, since the runway-anchored detectors replace that path entirely. But it is a genuine negative result, it would have been invisible in a two-configuration comparison, and it is recorded here rather than quietly reverted.

10.3 Off-block and on-block are limited by reception, as expected

Coverage of 2.6% and a bias of +744 seconds. The bias is not noise: sustained movement is detectable only once reception begins, which is typically well after the aircraft actually leaves the stand. Both numbers are reported as the finding rather than tuned away, and they are why T05 (pushback) was not built — it needs strictly more coverage than this.

10.4 Does it hold on a second period?

Both periods were run. What 2024 can check is narrower than 2025: no flight list exists for that period, and the runway, ring, airport and block detectors all need one, so 2024 exercises the phase and crossing families only. The ALDT and ATOT headline figures are therefore **single-period**.

For what it does cover, the two disagree in an instructive way.

Rung	landing coverage, 2025	2024
L01_clean_tracks	6.39%	6.66%
L02_smoothing	6.49%	0.96%
L03_complete_rules	0.69%	0.23%

Requiring complete fuzzy rules reproduces. A large relative collapse on both periods – 89% and 76% – so the finding is a property of the change rather than of a month.

The smoothing does not. Restoring the reference implementation’s 60-second majority vote is near-neutral on 2025 and costs 86% of landing coverage on 2024. This was the change with the strongest prior justification in the whole study, and it is the one the second period contradicts.

One confound has to be stated with it. The 2024 tracks predate step 02’s altitude repair, so that run substitutes the raw barometric altitude. The period therefore differs in input preparation as well as in date, and the smoothing operates on phase labels derived from exactly that column. The divergence may be the substitution rather than the month, and this study cannot separate the two. What it can say is that the change is **not robust across the conditions tested**, which is the useful half of the claim: a fix that behaves this differently on noisier input should not be adopted on the strength of its rationale alone.

10.5 What has still not been measured

The vertical family cannot be scored against data at all. No source available to OPDI holds level-segment truth. The claim made for KPI17 and KPI19 is conformance to ICAO’s published algorithm, tested on trajectories whose geometry is known by construction. That is checkable; “accurate” is not.

Top-of-climb and top-of-descent timing rests on no external evidence. The counts move — 114,099 to 100,925 for TOC — but nothing says which is closer to the truth.

The 2025 ladder is stale against the current code. It was produced before the changes that made the 2024 period runnable – the track redirect, the column backfills, the identity source. Those cannot alter a 2025 result, but the provenance fingerprint covers the file rather than the behaviour, so the chain reports it stale and this page says so rather than re-recording it as current. Clearing it honestly means re-running it.

Ring crossings are not yet scored against C40_CROSS_TIME. The detector runs and its volumes are now plausible, but the comparison against the reference crossing times is not in this version.

11 Breaking change for consumers

The new version does **not** re-emit `first-xing-fl*` and `last-xing-fl*`. Those types keep their meaning under the frozen `events_v0.0.2`, so published data stays reproducible, but consumers moving to the new version must migrate to the crossing sequence number carried in each event’s `info`: the first crossing is sequence 1, the last is the maximum.

Consumers should also know that `level-start/level-end` and the new ICAO `level-off-*` family answer different questions and are not interchangeable, and that the event table grows materially — every crossing is now emitted where previously only two per level were.

12 Provenance

Table 4

Output	Script	Git SHA	Produced
bridge_2024.json	benchmarks/events_gt_hoy	6054e82	2026-08-14
bridge_2025.json	benchmarks/events_gt_hoy	6054e82	2026-08-14
inventory_2024.csv	benchmarks/event_bench	71dcb643	2026-08-15
inventory_2025.csv	benchmarks/event_bench	71dcb643	2026-08-15
ladder_2024.csv	benchmarks/event_bench	6054e82	2026-08-15
ladder_2025.csv	benchmarks/event_bench	71dcb643	2026-08-15

Every figure above is produced by `benchmarks/regenerate_events.py`, which re-runs a job only when the source files that job depends on have changed. Outputs with no manifest entry are reported as unverified rather than shown as fact. The ladder outputs are absent from this table because they have not been produced.